

Unit II – Software Testing:

Q1. Explain the IEEE 829 Test Plan Format with its components.

The IEEE 829 standard provides a framework for organizing and documenting the process of software testing.

Components of IEEE 829 Test Plan:

1. Test Plan Identifier – Unique ID for test plan.
2. Introduction – Overview of the test plan, purpose, scope.
3. Test Items – Software modules/features to be tested.
4. Features to be Tested – List of functionalities covered.
5. Features Not to be Tested – Exclusions/Out-of-scope items.
6. Test Approach – Strategy used (manual/automated).
7. Pass/Fail Criteria – Benchmarks for test success.
8. Suspension/Resumption Criteria – When to pause and resume tests.
9. Test Deliverables – Artifacts like test scripts, defect reports.
10. Testing Tasks – Assigned roles and responsibilities.
11. Environmental Needs – Hardware/software/setup needed.
12. Responsibilities – Testers, developers, QA leads.
13. Schedule – Timeline of activities.
14. Risks and Contingencies – Possible failures and recovery.

EXAMPLE:

OVERVIEW

A test plan is a crucial written document that describes the testing strategy for a software or hardware project to ensure its functionality and reliability. It outlines the scope of testing, resources needed, the test environment, and the test cases that will be executed. It aims to ensure the systematic and coordinated execution of all necessary tests, thoroughly examining the product's functionality, reliability, and adherence to requirements.

What is a Test Plan?

A test plan is a detailed document that describes the approach, goals, resources, timeline, and spectrum of testing procedures needed to guarantee the performance and quality of a software product.

Example: Developing a test plan involves the contribution of the test lead, the test manager, and the test engineer.

Test Plan Example

To develop a test plan, we need to follow some steps in the testing process. Here is a test plan example for a hypothetical software application.

1. Introduction

This testing plan is for the E-Commerce application XYZ, version 1.0, which offers a wide range of products to customers and facilitates seamless online retail transactions. This testing aims to ensure that the web application meets the requirements and is free of defects.

2. Test Items

- Web Application: XYZ, version 1.0
- Build Number: 101

3. Features to be tested

- User login and registration
- User profile management
- Search and filtering functionality
- Shopping cart and checkout
- Payment gateway integration
- Email and SMS notifications
- Data export

4. Test Environment

- Operating System: Windows 11, MacOS
- Browser: Google Chrome, Firefox, Safari
- Hardware: Intel i5 processor, 8GB RAM
- Server: AWS

5. Test Schedule

- Test Planning: e.g., 15th April 2024 - 20th May 2024
- Test Case Development: e.g., 21st May 2024 - 25th June 2024
- Test Execution: e.g., 26th June 2024 - 5th July 2024
- Test Closure: e.g., 6th July 2024

6. Test Deliverables

- Test cases
- [Test scripts](#)
- [Test reports](#)
- Defect reports
- Performance test report
- Responsive test report

7. Test Responsibilities

Job	Stakeholders Involved
Write Test Plan	Test Lead, Test Manager, Test Engineer
Review Test Plan	Test Lead, Test Manager, Test Engineer, Customer, Development Team
Approve Test Plan	Customer, Test Manager
Write Test Case	Test Lead, Test Engineer
Review Test Case	Test Engineer, Test Lead, Customer, Development Team
Approve Test Case	Test Manager, Test Lead

Execute Test	Test Lead, Test Engineer
Maintain Testing Environment	Server Administrator
Fix Defects and Provide Support	Developer

8. Test Approach

- [Manual testing](#) can be used to test all the functionalities of the web application.
- [Automated testing](#) can be used to test the performance and load of the web application.
- [Responsive testing](#) will be done to ensure the web application is compatible with different devices and screen sizes.

9. Exit Criteria

- All the identified defects must be fixed and verified.
- All the test cases must be executed and passed.
- All the test deliverables must be completed and submitted.
- [Performance tests](#) should pass the threshold limit.

This is a sample test plan; it may vary depending on the complexity of the web application and the organization's testing process.

Test Plan Template

- 1) Test Plan Identifier
- 2) References
- 3) Introduction
- 4) Test Items
- 5) Software Risk Issue
- 6) Features to be Tested
- 7) **Features not to be Tested**
- 8) Approach
- 9) Item Pass/Fail Criteria
- 10) Entry & Exit Criteria

www.softwaretestingvideos.com

Test Plan Components



Q2. Define a Test Scenario and provide an example.

A Test Scenario is a high-level documentation of what to test. It represents a real-world use case.

Example:

Feature: Login functionality

Test Scenario: Verify that users can log in with valid credentials.

EXAMPLE IN DETAIL STEP BY STEP:

What Are the Test Cases for the Login Page?

Test cases for the login page help validate user input, enforce security rules, and ensure reliable access control. They focus on scenarios like empty fields, invalid formats, weak passwords, and login attempt limits.

Tips for Writing Better Test Cases for Login Page

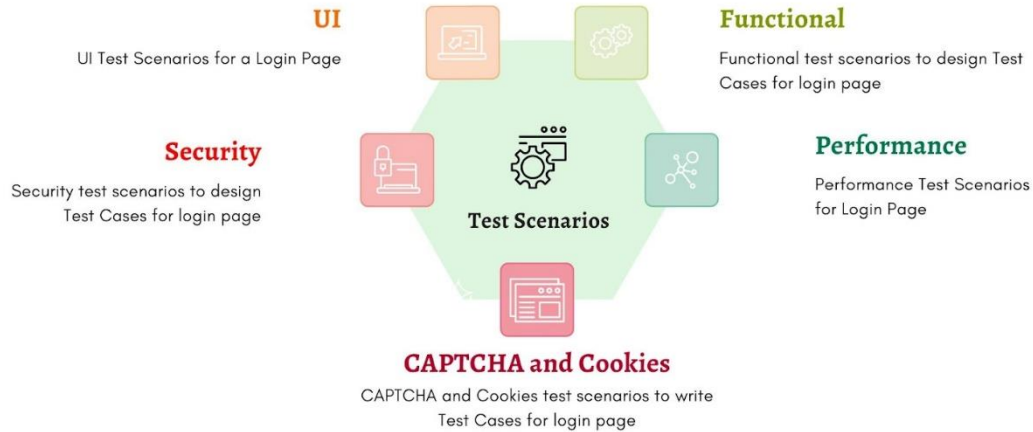
- Identify key functionalities and risk areas before writing test cases.
- Focus each test case on a single functionality for easier tracking and debugging.
- Include clear instructions, required test data, preconditions, and expected outcomes.
- Cover both positive and negative scenarios to test expected and unexpected behaviors.
- Prioritize test cases based on risk and criticality.
- Ensure test cases are independent and do not affect one another.
- Use both valid and invalid inputs for thorough validation.
- Update test cases whenever the application changes.
- Automate repetitive test cases to save time and increase coverage.
- Use concise, meaningful names and document each test case properly.

What Are the Test Cases for the Login Page?

Test Cases for the login page are crucial to ensure system security and user experience. These include validating empty fields, handling incorrect formats (e.g., special characters), enforcing password strength, and limiting login attempts. For example, testing password strength ensures users create secure credentials. Overall, these test cases are vital to uncovering potential vulnerabilities, guaranteeing a robust login mechanism, and enhancing the application's overall security. See here for more [examples of web application test cases](#).

The below image shows the various login page test scenarios that are essential for quality testing.

Test Scenarios for a Login Page



Functional Test Cases of Login Page

Test Case Type	Test Cases
Positive	– Successful login with valid
	– Login with a valid username and case-insensitive
	– Successful login with a remembered username and password
	– Successful login after password
	– Login from multiple devices
	– Successful login using a social media account
	– Login with a valid username and password within specified character length
Negative	– Successful login with special characters in the password.
	– Unsuccessful login with an invalid username and
	– Login with a blank username and password. Unsuccessful login with an empty password
	– Login attempt with an account locked due to multiple unsuccessful login attempts
	– Login with a valid username and an incorrect case-sensitive password
	– Unsuccessful login with an account under review or pending verification
	– Login attempt with a deactivated or terminated user account.

Non-functional Test Cases for Login Page

Test Case Type	Test Cases
Positive	– Implementation of HTTPS to ensure secure data transmission
	– Session timeout functionality for automatic logout after inactivity
	– Implementation of account lockout after a specified number of unsuccessful login attempts
	– Captcha verification for preventing automated bot logins

Negative	– Password complexity requirements to ensure strong user
	– Secure handling of password reset
	– User account activity monitoring for detecting suspicious behavior.
	– The absence of HTTPS leading to insecure data
	– Lack of session timeout, posing a risk for unauthori
	– Unencrypted storage of passwords, exposing sensitive
	– Weak or no password complexity
	– Absence of multi-factor authentication or CAPTCHA, reducing lo
	– Lack of user account activity monitoring and missing detection of suspicious behavior.

Check here – [Logout Testcases](#)

UI Test Scenarios for Login Page

Simply listing out and knowing the functional and [non-functional test cases](#) is not sufficient. As a tester, you must also understand the various test cases that impact user behavior and satisfaction once they start using the application. Here are some of the [UI test scenarios](#) for the login page:

Verify Field Validations:

- Check if the username and password input fields are present on the page and accept valid credentials.
- Check if the login is successful after changing the password.
- Test the system’s behavior when entering more characters than the allowed limit for both username and password.
- Validate that the system restricts login attempts after reaching a defined limit and displays an appropriate message.

Verify Error Messages:

- Input fields should display appropriate validation messages when both username and password are empty.
- Ensure the system validates and displays an error for an invalid username format (e.g., special characters).
- Validate the system’s response to an invalid password format, triggering an error message.

Verify the Functionality of the “remember Password” Checkbox:

- Verify that the “*Remember Password*” checkbox retains its selected/unselected state after the page reloads.

- Confirm that selecting the checkbox and logging in stores user credentials for future sessions.
- Test the checkbox functionality across different browsers to ensure consistency
- Ensure that sensitive information, such as passwords, is not compromised when using the “*Remember Password*” feature.

Verify the “forgot Password” Functionality:

- Confirm the presence and functionality of the “*Forgot Password*” link.
- Validate that users receive a prompt or instructions on how to reset their password.
- Test the process of receiving and verifying password reset emails.
- Confirm that clicking the reset link directs users to a secure page for password reset.
- Test the system’s response when entering an invalid or unregistered email for password reset.
- Confirm that users receive confirmation messages after successfully resetting their passwords.

Verify the “create an Account” Link:

- Confirm the presence and visibility of the “*Create an Account*” link on the login page.
- Validate that clicking the link directs users to the account creation page or sign-up page.
- Check if users receive appropriate prompts or guidance when attempting to create an account.
- Ensure the “*Create an Account*” link is appropriately positioned and easily noticeable.

Verify the Visibility of Elements:

- Confirm the visibility of the username and password input fields.
- Ensure the visibility and clarity of the login button.
- Verify the visibility and accessibility of the “*Remember Password*” checkbox.
- Confirm the visibility of the “*Forgot Password*” link.
- Test the visibility and clarity of error messages for incorrect login attempts.

Verify the Responsiveness of the UI:

- Test the login page’s responsiveness on different screen sizes, including desktop, tablet, and mobile.

- Ensure consistent visibility and functionality across various web browsers.
- Validate the UI's responsiveness to changes in device orientation (landscape/portrait).
- Confirm that text and font sizes adjust appropriately for different screen sizes.
- Check that buttons and elements maintain proper placement and spacing on different devices.

Verify Compatibility with Different Browsers:

- Confirm that the login page displays correctly and functions as expected on the latest version of Google Chrome, Mozilla Firefox, Apple Safari, Microsoft Edge, and Internet Explorer.
- Check compatibility across different versions of each browser to cover a broader user base.
- Validate that the UI elements and functionalities remain consistent across all tested browsers.
- Confirm that the login page maintains its responsive design across various browsers and their different versions.
- Check for consistent font rendering across browsers, avoiding distortions or display issues.
- Validate the compatibility of CSS to ensure uniform styling across browsers.

Performance Test Cases for Login Page

Ensuring optimal performance for a login page is crucial for providing a seamless user experience. Here's a list of [performance test cases](#) to validate the efficiency of the login process:

- Verify how the login page handles a specified number of concurrent users attempting to log in simultaneously.
- Measure and validate the time taken for the login page to respond to user inputs under normal and peak load conditions.
- Evaluate the number of successful logins the system can handle within a defined time frame to ensure efficient throughput.
- Evaluate the system's stability over an extended period, checking for memory leaks or performance degradation during continuous usage.
- Test the performance when users attempt to log in with incorrect credentials, ensuring the system can handle authentication failures efficiently.

- Simulate different network conditions to assess how the login page performs under varying latency scenarios.
- Monitor the utilization of system resources (CPU, memory, etc.) during login operations to identify potential bottlenecks.
- Verify login performance across various browsers to ensure consistency and optimal user experience on different platforms.

Login Page Test Cases – CAPTCHA & Cookies

Below are some CAPTCHA and Cookies test scenarios to write Test Cases for the login page:

CAPTCHA:

- Verify whether the CAPTCHA is present on the login page.
- Verify if the CAPTCHA regenerates when required.
- Check if there is an option to refresh the CAPTCHA image for users having difficulty with the current one.
- Check if the CAPTCHA validates the input and prevents the login for invalid CAPTCHA.
- Verify that the CAPTCHA has a reasonable timeout period to prevent expiration during user interaction.
- Test if CAPTCHA is accessible via alternative text or audio options.

Check here- [testcases for captcha](#)

Cookies:

- Verify that the login page sets a cookie upon successful login.
- Verify that the cookie lasts across multiple browser sessions and remains valid until it expires.
- Verify that the cookie deletes upon logout/upon session expiry.
- Verify that the cookie is securely encrypted to prevent unauthorized access.

Check here – [Cookie Testing](#)

Q3. What is a Test Case? Write an example for ATM withdrawal.

A Test Case is a set of conditions or inputs used to test a specific function.

Example:

Test Case ID: TC_001

Title: ATM Cash Withdrawal

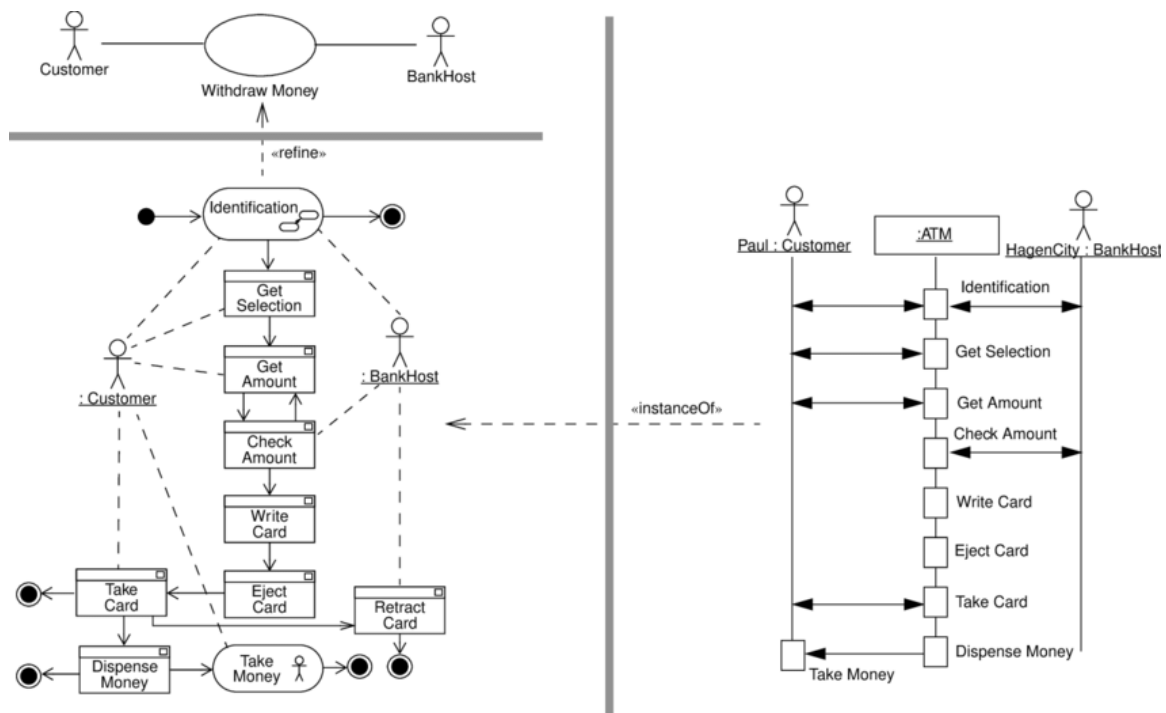
Pre-condition: User has a valid ATM card

Steps:

1. Insert card
2. Enter PIN
3. Choose Withdraw
4. Enter amount
5. Confirm

Expected Result: Amount should be dispensed

Post-condition: Balance updated



Q4. Differentiate between Positive and Negative Test Cases.

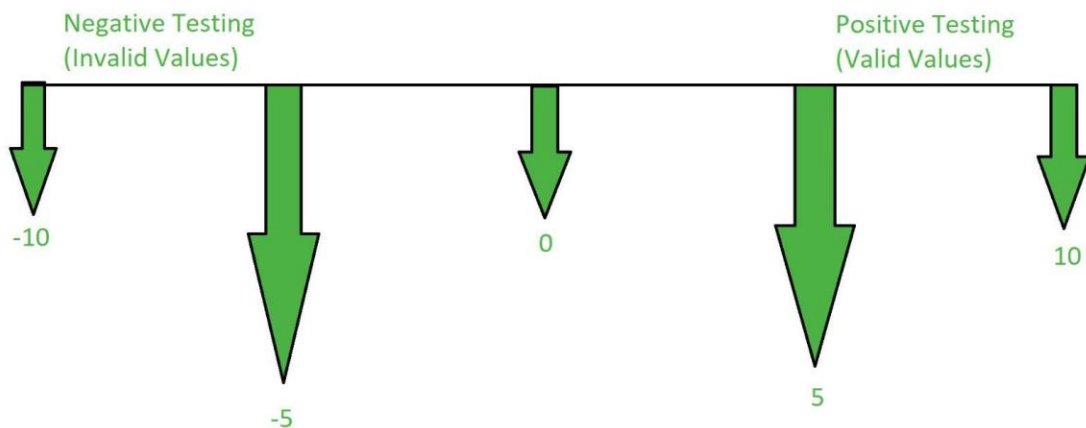
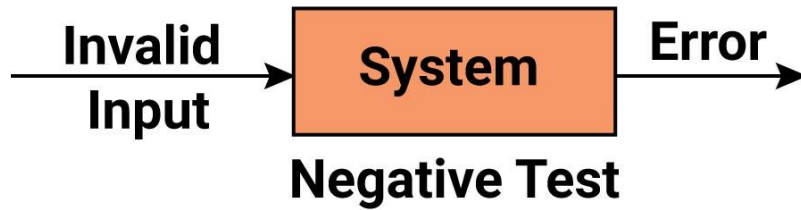
Type	Positive Test Case	Negative Test Case
------	--------------------	--------------------

-----	-----	-----
-------	-------	-------

Definition	Checks valid input & expected output	Checks system with invalid input
------------	--------------------------------------	----------------------------------

| Example | Logging in with correct credentials | Logging in with empty username |

| Goal | Ensure the system works as expected | Ensure system handles errors properly |



Q5. Write 3 positive and 3 negative test cases for user registration.

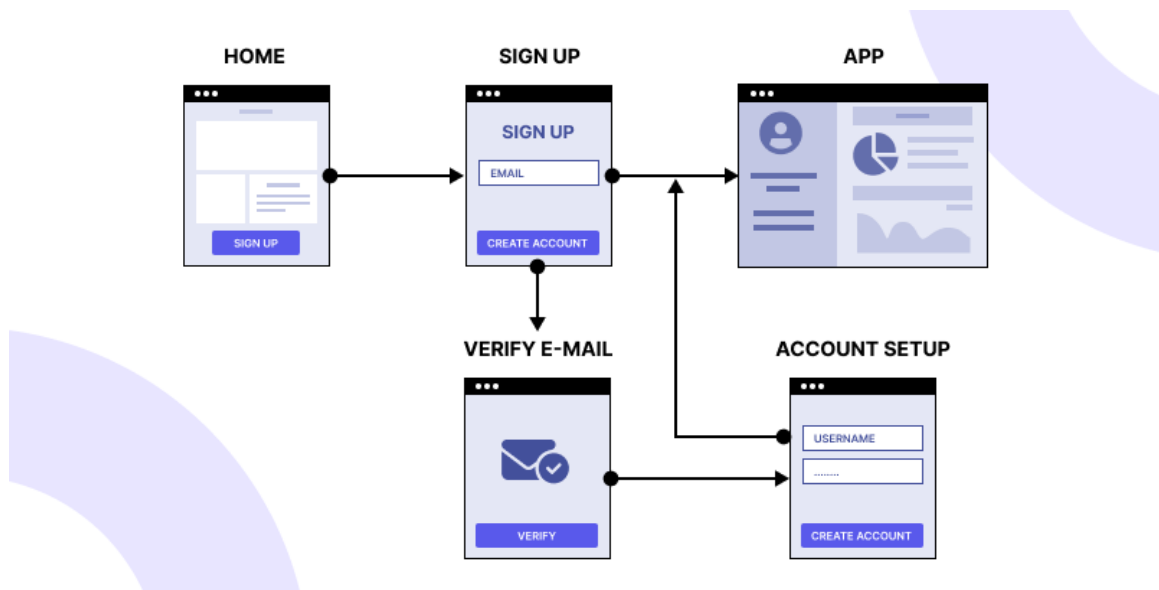
Positive Test Cases:

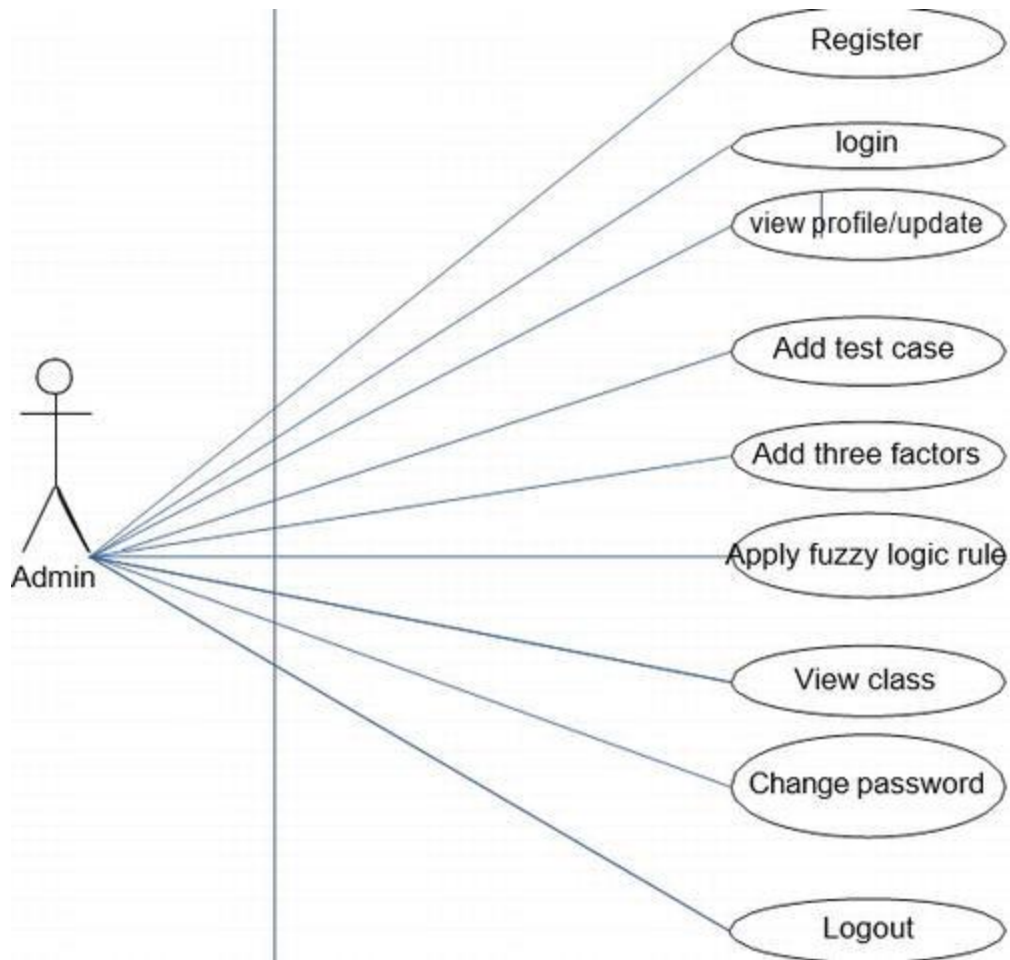
1. Valid email, strong password, and unique username.
2. Password and confirm password match.
3. Mobile number is valid and OTP is verified.

Negative Test Cases:

1. Email left blank.
2. Passwords do not match.
3. Username already exists.

Test Case Id	Description	Expected Out put	Actual Output	Pass/ Fail	Figure No
1	User ID and password are blank	"Enter the User id and Password" Error message should be display	message displayed	Pass	4
2	User ID is blank	"Enter the User id" Error message should be display	message displayed	Pass	5
3	Password is blank	"Enter the Password" Error message should be display	message displayed	Pass	6
4	Wrong ID	"This ID is invalid" Error message should be display	message displayed	Pass	7
5	Wrong Password	"Password is incorrect. Please enter correct Password " Data should save in the database	message displayed	Pass	8
6	Cancelled User ID	" Your ID Was Cancelled Please Contact librarian" Blank data should be display	message displayed	Pass	9
7	Correct User ID And Password	The main form should be display and login history Data should save in the database	The main form displayed and Saved in database	Pass	
8	Press Forget "Password Button"	Forget Password Form Should Be Display	Forget Password Form Displayed	Pass	

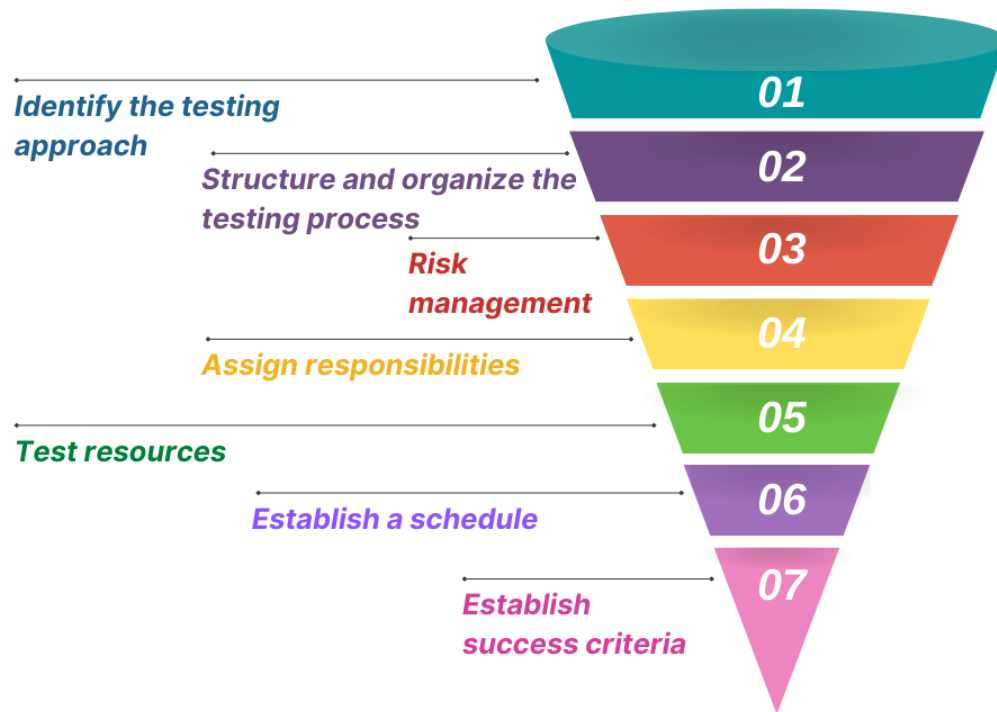




Q6. List and explain 5 key points to be considered while creating a test plan.

1. Scope of Testing – What will and won't be tested.
2. Resources Required – Team, tools, environments.
3. Risk Management – Known risks and mitigation strategies.
4. Entry and Exit Criteria – When to begin/stop testing.
5. Test Strategy – Black-box/white-box, manual/automated.

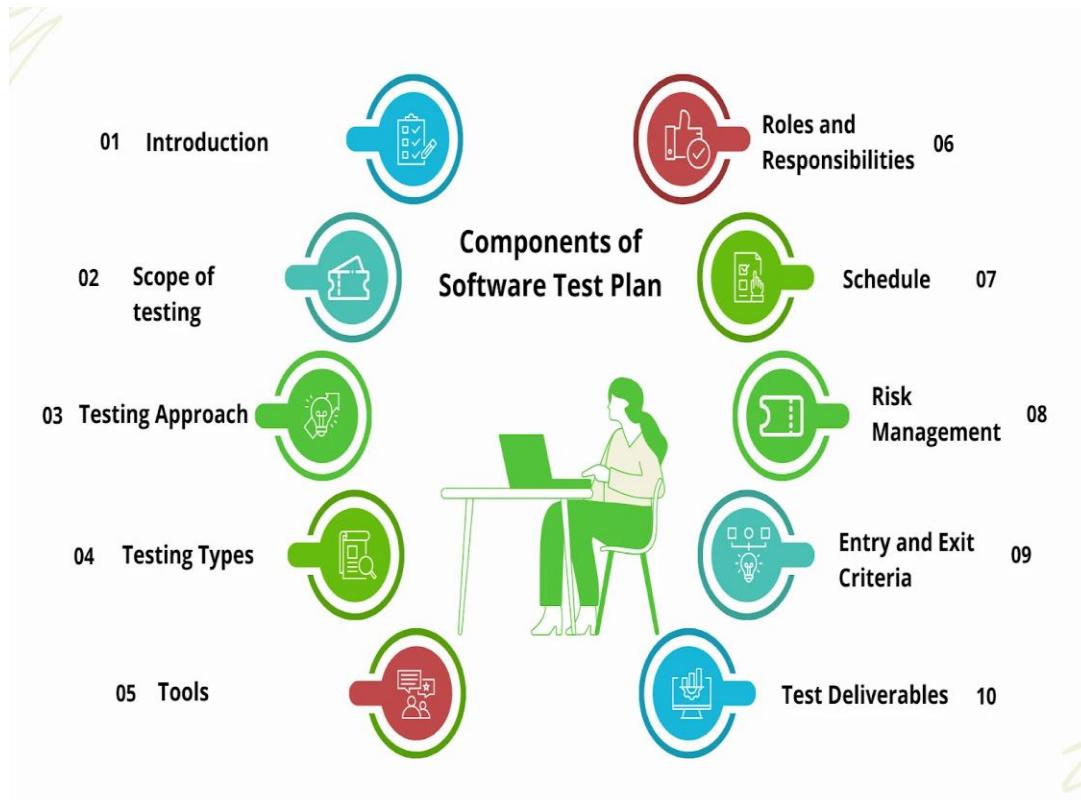
OBJECTIVES OF A SOFTWARE TEST PLAN



Or

- **Identify the testing approach:** Specify the type of the testing to be done. Figure out the important features and functions to be tested. The likelihood of reducing defects in the software increases when an appropriate testing approach is chosen, ensuring that critical components are tested against their specific requirements.
- **Structure and organize the testing process:** Clearly define the scope of the test: what is to be in scope and out of scope. It must outline clear directions toward the project goals. The sequence of the testing activities should be organized in a logical and structured flow. Appropriate organization will streamline it, hence improving the whole process.
- **Risk management:** This entails the identification of potential risks and stating appropriate solutions. It also involves strategies for dealing with issues that arise during the testing process.
- **Assign responsibilities:** Identify who will carry out each test so that all the tasks are clearly allocated. Defined roles decrease confusion and raise team efficiency.
- **Test resources:** Outline the resources required for testing, including tools such as test environments, hardware, software, and configurations. This ensures testing aligns closely with the conditions in which the software will be used.

- **Establish a schedule:** Establish clear milestones and deadlines as to when these test processes will be executed. This helps ensure testing activities are performed within the timeline of the project. It will also keep the test team on their toes to make sure everything is tested right on schedule.
- **Establish success criteria:** Clearly establish what constitutes success, which could be a certain passed number in testing or performance standard. This will ensure that the audience knows what's considered a successful test.



BEST PRACTICES FOR SOFTWARE TEST PLAN



CONCLUSION:

Conclusion

Creating a software test plan makes sure your entire development process leads to a product that works well and meets user needs. In this guide we have looked at what is a test plan and step by step process to create it. A good test plan helps you spot risks early, reduce bugs, and improve teamwork. In the end, this means delivering software that runs smoothly and gives users a great experience.

Think of your test plan as a living document. As your project changes, your test plan should adapt too. Regular updates based on feedback, new requirements, or lessons learned keep your testing relevant and effective.

By making your test plan flexible and focused on the user, you're setting your project up for success. Remember, testing isn't something you do once and forget about. It's a continuous process of improvement. Stay open to change, aim for quality, and make sure your software delivers real value to the people who use it.

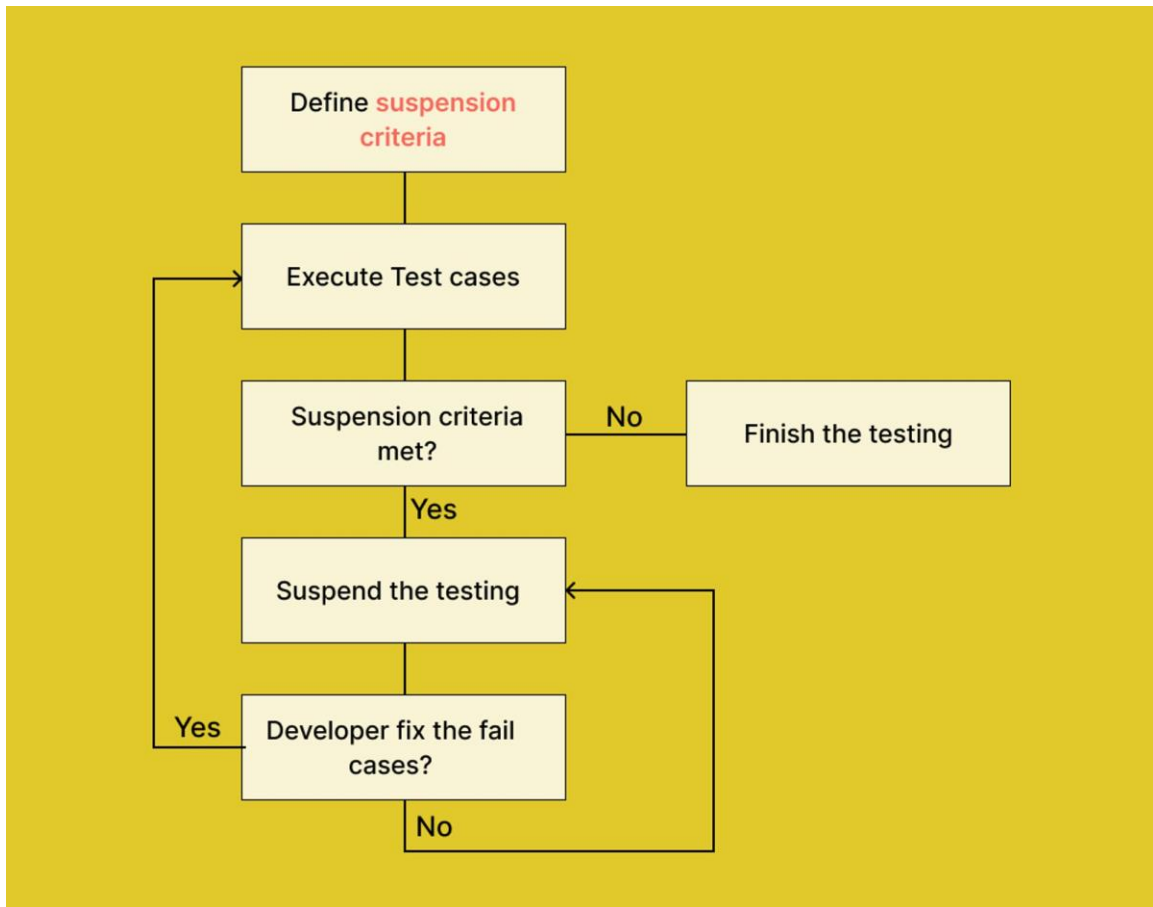
Q7. What is the importance of Suspension and Resumption Criteria in testing?

These criteria define when testing should pause (suspend) and resume, ensuring:

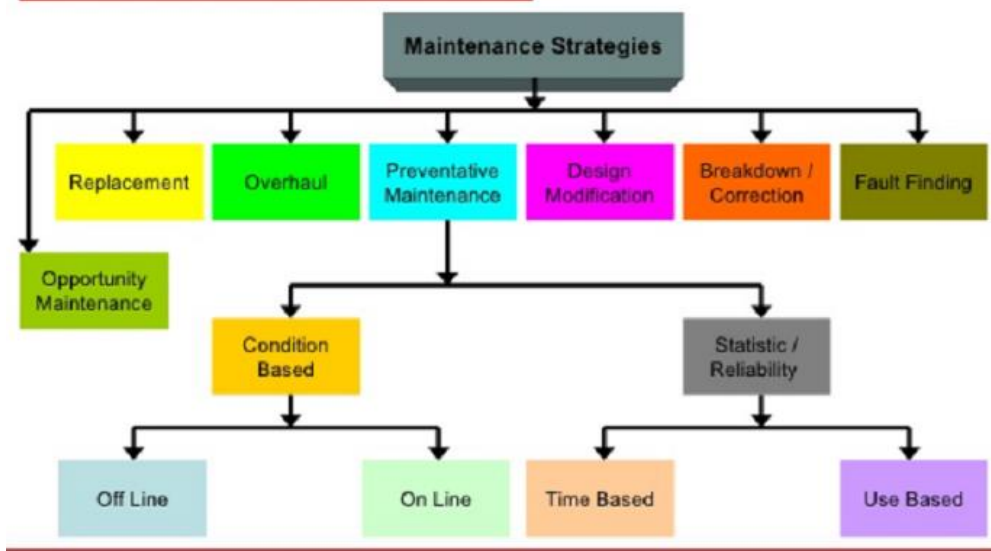
- Unstable builds are not tested.
- Downtime doesn't result in data loss.
- Retesting starts with known checkpoints.

Example:

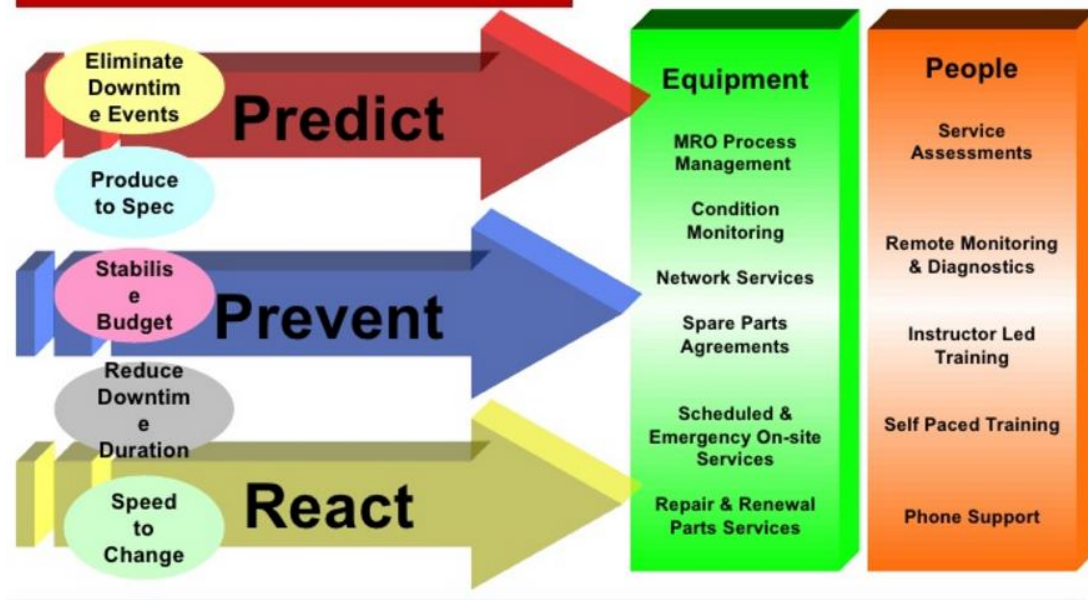
- Suspend if: Login module crashes repeatedly.
- Resume when: Login module fix is deployed and passes sanity testing.



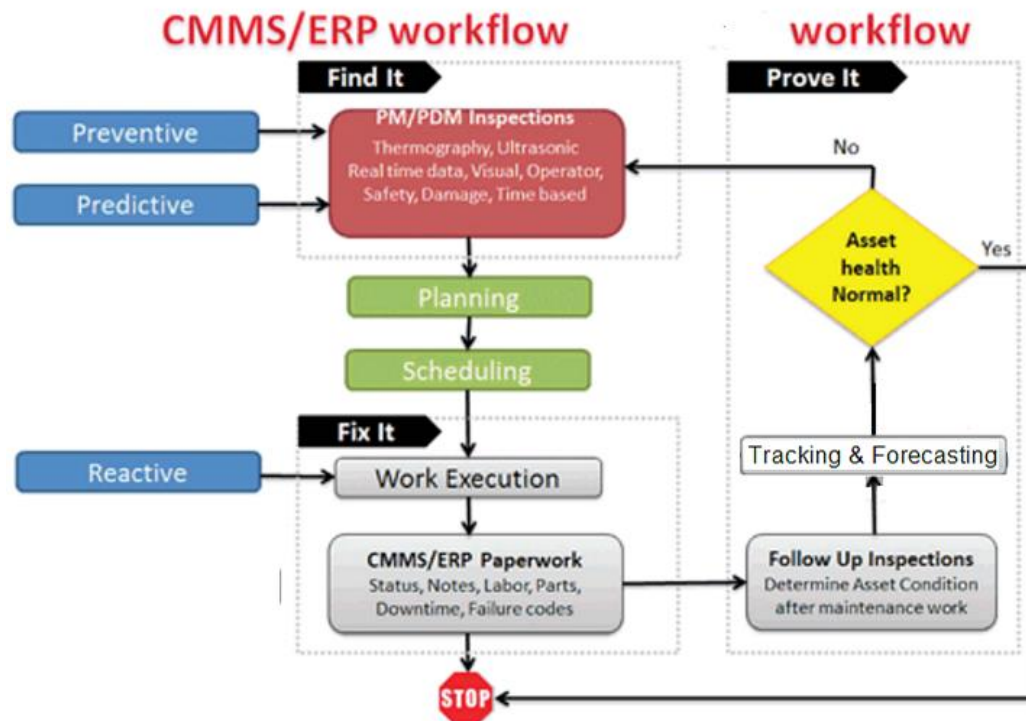
Maintenance Strategies



Link Strategies to Maintenance



Condition-Based Maintenance Workflow

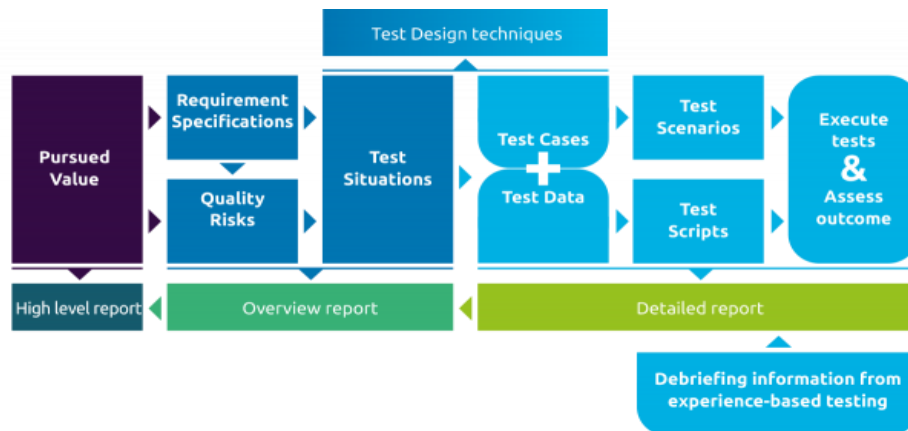


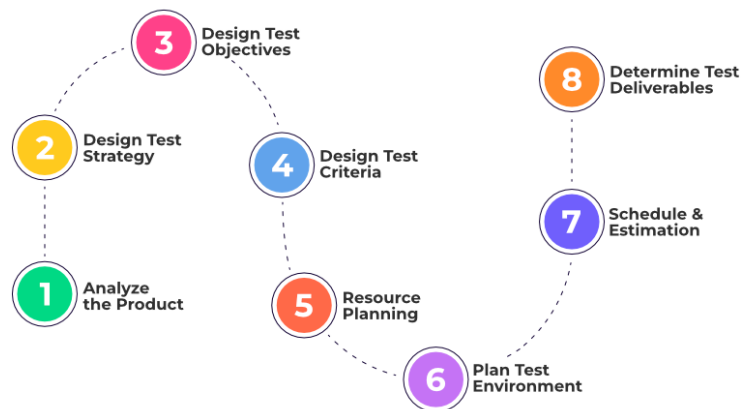
Q8. Explain how Test Deliverables are documented in a test plan. Give examples.

Test Deliverables are the documents and outputs generated during and after testing.

Examples:

- Test Plan Document – Outlines the entire testing strategy.
- Test Cases/Scenarios – Steps to be executed.
- Defect Reports – Bugs and issues logged.
- Test Summary Report – Overall results.
- Traceability Matrix – Requirement to test case mapping.





Q9: Explain the significance of the "Features Not to be Tested" section in a Test Plan.

A3: The "Features Not to be Tested" section is crucial for several reasons:

- **Scope Definition:** It explicitly defines the boundaries of the testing effort, preventing misunderstandings and scope creep.
- **Expectation Management:** It manages stakeholder expectations by clearly stating what will *not* be covered, avoiding the assumption that all features are implicitly included.
- **Resource Allocation:** By excluding certain features, it helps in optimizing resource allocation and focusing efforts on the defined scope.
- **Risk Mitigation:** It can highlight features that are intentionally excluded, possibly due to low risk, out-of-scope requirements, or being covered by other testing phases/teams. This transparency can help in identifying potential gaps or agreements.

Q10: How does adherence to the IEEE 829 Standard benefit a software development project in the long run?

A10: Adherence to the IEEE 829 Standard benefits a software development project in the long run by:

- **Improved Communication:** Provides a common language and structure for test documentation, reducing ambiguity and improving understanding among teams and stakeholders.

- **Enhanced Quality:** Leads to more thorough and systematic testing, which helps in identifying defects earlier and improving the overall quality of the software.
- **Increased Efficiency:** Standardized documentation makes it easier to onboard new team members, understand past testing efforts, and avoid redundant work.
- **Better Maintainability:** Test documents are easier to maintain and update as the software evolves.
- **Greater Reusability:** Test plans, cases, and other artifacts can be reused across different projects or releases, saving time and effort.
- **Reduced Risk:** Clear documentation of scope, risks, and contingencies helps in proactive risk management.
- **Legal and Regulatory Compliance:** Provides a structured approach that can aid in meeting certain compliance requirements.
- **Improved Accountability:** Clearly defined responsibilities and deliverables enhance accountability within the testing team and across the project.
- **Knowledge Transfer:** Serves as a valuable knowledge base for future projects and teams.